# kernel 1/4




Uncle Linus Wants You!

# Le noyau Linux

Caractéristiques principales

- Portable
- Polyvalent
- Stable
- Mature
- Sécurisé
- Robuste

# Qui est derrière Linux ?

- Une communauté open source
- Environ 1200 développeurs par version
- Environ 200 mainteneurs
- Environ 80 % des modifications sont sponsorisées
- Linus Torvalds effectue les releases officielles

# Utilisation et support matériel

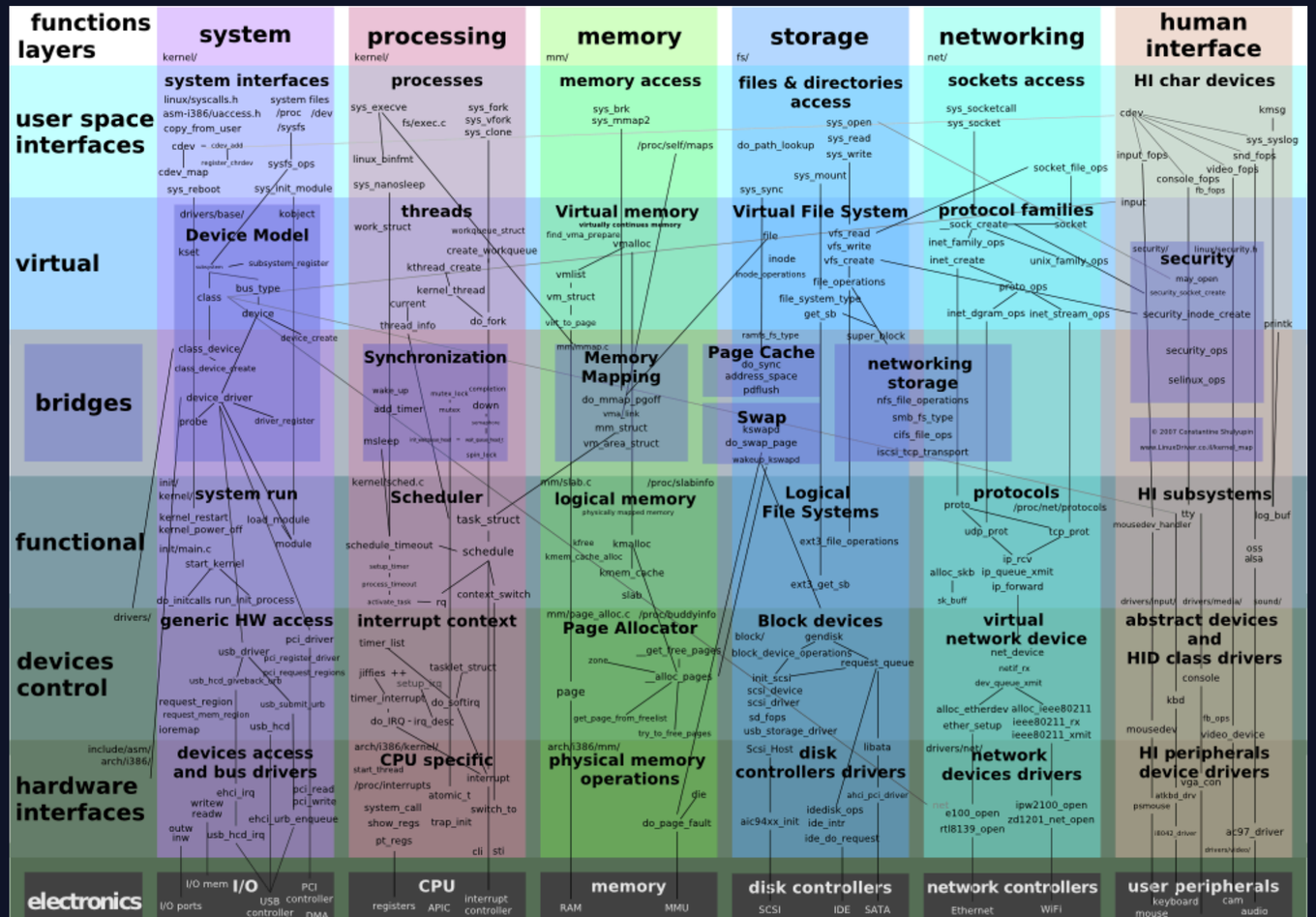
Linux supporte plus d'architectures matérielles que tout autre OS.

https://en.wikipedia.org/wiki/Usage_share_of_operating_systems

- Smartphone : 72 % (android/linux)
- Tablet: 48 % (android/linux)
- Desktop/Laptop : 4 % (linux), 71 % (windows)
- Serveur (web) : 63 % (linux)
- Top 500 Supercomuters : 100 % (linux)

# Découvrir Linux

## Vue d'ensemble



# Les sources


The Linux Kernel Archives — Mozilla Firefox
The Linux Kernel Archives
https://kernel.org
The Linux Kernel Archives
About Contact us FAQ Releases Signatures Site news
Protocol Location
HTTP https://www.kernel.org/pub/
GIT https://git.kernel.org/
RSYNC rsync://rsync.kernel.org/pub/
Latest Release
6.11
mainline: 6.11 2024-09-15 [tarball] [pgp] [patch] [view diff] [browse]
stable: 6.10.10 2024-09-12 [tarball] [pgp] [patch] [inc. patch] [view diff] [browse] [changelog]
longterm: 6.6.51 2024-09-12 [tarball] [pgp] [patch] [inc. patch] [view diff] [browse] [changelog]
longterm: 6.1.110 2024-09-12 [tarball] [pgp] [patch] [inc. patch] [view diff] [browse] [changelog]
longterm: 5.15.167 2024-09-12 [tarball] [pgp] [patch] [inc. patch] [view diff] [browse] [changelog]
longterm: 5.10.226 2024-09-12 [tarball] [pgp] [patch] [inc. patch] [view diff] [browse] [changelog]
longterm: 5.4.284 2024-09-12 [tarball] [pgp] [patch] [inc. patch] [view diff] [browse] [changelog]
longterm: 4.19.322 2024-09-12 [tarball] [pgp] [patch] [inc. patch] [view diff] [browse] [changelog]
linux-next: next-20240916 2024-09-16 [browse]
Other resources
Git Trees
Patchwork
Mirrors
Documentation
Wikis
Linux.com
Kernel Mailing Lists
Bugzilla
Linux Foundation
Social
Site Atom feed
Releases Atom Feed
Kernel Planet
05:20

# L’arborescence du noyau (1/3)

- `arch/` Code spécifique à l'architecture
- `block/` Couche d'E/S bloc
- `crypto/` API cryptographique
- `Documentation/` Documentation du code source du noyau
- `drivers/` Pilotes de périphériques (sauf audio)
- `firmware/` Micrologiciel pour certains périphériques
- `fs/` Infrastructure des systèmes de fichiers
- `include/` En-têtes du noyau
  - `include/linux/` En-têtes du noyau principal
  - `include/uapi/` En-têtes de l'API espace utilisateur
- `init/` Initialisation du noyau
- `ipc/` Communication interprocessus System V (sémaphores, mémoire partagée, files de messages)

# L’arborescence du noyau (2/3)



- `kernel/` Noyau Linux principal
- `lib/` Bibliothèque de routines (listes, arbres, chaînes, etc.)
- `mm/` Gestion de la mémoire
- `net/` Code réseau (pas les pilotes !)
- `samples/` Exemples de code (trace, kobject, kprobes)
- `scripts/` Scripts pour usage interne ou externe
- `security/` Frameworks de sécurité
- `sound/` Code audio et pilotes son
- `tools/` Outils espace utilisateur
- `usr/` Génère l'initramfs
- `virt/` Infrastructure de virtualisation (kvm)

## L’arborescence du noyau (3/3)

- `COPYING` La licence du noyau (GPLv2)
- `CREDITS` Les personnes ayant contribué au noyau
- `Kbuild` Système de compilation du noyau
- `Kconfig` Description des paramètres de configuration
- `MAINTAINERS` Mainteneurs des sous-systèmes et pilotes
- `Makefile` Makefile de base du noyau
- `README` Vue d'ensemble du noyau
- `REPORTING-BUGS` Instructions pour signaler les bugs

## Processus de développement

- Chaque fichier dispose d'un ensemble de mainteneurs.
- Ils sont responsables du triage des bugs, de la revue des patches et de la direction des modifications.
- Les patches sont envoyés aux listes de diffusion pour revue.
- Une fois approuvés, ils sont envoyés au mainteneur du sous-système.
- Les mainteneurs de sous-systèmes les examinent et les placent dans une branche spéciale.
- Ils envoient cette branche à Linus Torvalds, qui la fusionnera dans sa branche.

## Cycle de release

- La fenêtre de fusion est ouverte (~2 semaines).
- Des candidats à la release ( `-rc` ) sont publiés.
- Aucune nouvelle fonctionnalité n'est ajoutée, seulement des corrections de bugs (~6-10 semaines).
- La release finale est taguée par Linus Torvalds.

## Configurer le noyau



- Le noyau est un fichier unique, résultat du processus de compilation.
- Des options à la compilation (flags `-D`) permettent de choisir quelles fonctionnalités sont intégrées et leurs paramètres.
- Cependant, après le démarrage, il peut charger des modules depuis le système de fichiers à l'exécution pour étendre ses fonctionnalités. Chaque module est un fichier unique.
- Configurer le noyau consiste à :
    - Choisir ce qui va dans le fichier principal et ce qui sera compilé en modules.
    - Configurer diverses options.

## Options du noyau

- Les options ont la forme `CONFIG_FEATURE` et un type, p.ex.
  - `CONFIG_MODULES` booléen (true/false)
  - `CONFIG_INITRAMFS_ROOT_UID` entier
  - `CONFIG_INITRAMFS_SOURCE` chaîne
  - `CONFIG_MAGIC_SYSRQ_DEFAULT_ENABLE` hexadécimal
  - `CONFIG_BTRFS_FS` tristate (true/module/false)
- Les options peuvent dépendre d'autres options.
- Deux types de dépendances :
  - A `select` B : activer A active B
  - A `depends on` B : A n'est pas visible tant que B n'est pas activé

# Kconfig

```
config BTRFS_FS
    tristate "Btrfs filesystem support"
    select CRYPTO
    select CRYPTO_CRC32C
    select ZLIB_INFLATE
    select ZLIB_DEFLATE
    select LZO_COMPRESS
    select LZO_DECOMPRESS
    select RAID6_PQ
    select XOR_BLOCKS

    help
      Btrfs is a general purpose copy-on-write filesystem
      with extents, writable snapshotting, support for
      multiple devices and many more features focused on
      fault tolerance, repair and easy administration.

      [...]
```

## Fichiers de configuration

`.config`

Fichiers texte simples, format `clé=valeur`.

Fichiers `.config` par défaut

- `make defconfig` : nouvelle configuration avec les valeurs par défaut du defconfig fourni par $ARCH
- `make x86_64_defconfig` : demande le defconfig d'une plateforme

## Éditer les fichiers `.config`

- `make config` : interface texte
- `make menuconfig`, `make nconfig` : interface menu/ncurses
- `make xconfig`, `make gconfig` : interface graphique
- `make oldconfig` : met à jour `.config` avec les options de la nouvelle version

## Construxtion de Noyaux



Les distributions Linux activent généralement de nombreuses fonctionnalités et pilotes du noyau, la plupart compilés en modules.

Lire le fichier de configuration de votre noyau

Nécessite `CONFIG_IKCONFIG_PROC=y` :

```
$ zcat /proc/config.gz
CONFIG_64BIT=y
CONFIG_X86_64=y
CONFIG_X86=y
CONFIG_INSTRUCTION_DECODER=y
CONFIG_OUTPUT_FORMAT="elf64-x86-64"
CONFIG_ARCH_DEFCONFIG="arch/x86/configs/x86_64_defconfig"
...
```

## Compilation



- N'interagir qu'avec le Makefile du répertoire racine

```
$ make defconfig # Éditer .config
$ make # Compiler le noyau et les modules
```

Produit :

- …
- `vmlinux` : objet ELF du noyau, non démarrable
- `arch/x86/boot/bzImage` : image compressée démarrable du noyau
- `**/*.ko` : Modules

## Autres cibles

- `make help` : liste toutes les cibles disponibles
- `make modules` : compile/recompile les modules
- `make headers_install` : « installe » les en-têtes dans le dossier local `usr/`
- `make modules_install` : installe dans `/lib/modules/KVER`
  - `INSTALL_MOD_PATH=dir/` pour choisir le répertoire

# Démarrage du système

# Séquence de démarrage (1/2) — Jusqu'au noyau

1. BIOS / UEFI
   Initialise le matériel, charge le bootloader
   depuis le MBR ou la partition EFI (GPT)

2. Bootloader  (GRUB, syslinux…)
   Charge le noyau compressé  (bzImage / vmlinuz)
   et l'initramfs en mémoire
   Transmet la « ligne de commande noyau »

3. Noyau Linux
   Se décompresse et s'initialise
   Détecte le matériel, monte l'initramfs sur /
   Exécute /init (ou le programme indiqué par
   init= sur la ligne de commande)

# Séquence de démarrage (2/2) — Depuis init

```
4. init (PID 1) — dans l'initramfs
   Monte les pseudo-systèmes de fichiers
   Prépare le vrai rootfs (détection, fsck…)
   Pivote la racine vers le rootfs définitif
   (pivot_root ou switch_root)

5. init définitif (systemd, OpenRC, BusyBox init…)
   Démarre les services, les démons, le login
```

*Dans nos TPs : étape 1: le noyau reste sur l'initramfs et le /init est notre script shell.*

# L’initramfs



- Archive **cpio** compressée (gzip, lz4…) embarquée dans la mémoire au démarrage.
- Le noyau l'extrait sur `tmpfs` et exécute `/init`.
- Contient le **minimum** pour démarrer : busybox, quelques modules, `init`.
- Avantages : pas de dépendance au système de fichiers hôte, entièrement en RAM.

```
# Créer l'image cpio gzippée
find . -print0 | cpio --null -ov --format=newc | gzip -9 > initramfs.cpio.gz

# Inspecter son contenu
zcat initramfs.cpio.gz | cpio -tv
```

# Ligne de commande du noyau



Passée par le bootloader, visible dans `/proc/cmdline`.

| Paramètre | Rôle |
|---|---|
| `root=/dev/sda1` | Périphérique rootfs définitif |
| `root=/dev/nfs` | Rootfs via NFS |
| `init=/sbin/init` | Programme init (défaut : `/init`) |
| `console=ttyS0` | Console série (QEMU `-nographic`) |
| `console=tty0` | Console VGA standard |
| `rw` / `ro` | Rootfs en lecture-écriture / lecture seule |
| `quiet` | Réduit les messages de boot |
| `nomodeset` | Désactive KMS (utile en VM) |

# QEMU — options essentielles

```
# Boot avec initramfs (pas de disque)
qemu-system-x86_64 \
  -kernel arch/x86/boot/bzImage \
  -initrd initramfs.cpio.gz \
  -append "console=ttyS0" \
  -nographic

# Boot depuis une image disque avec GRUB
qemu-system-x86_64 -hda disk.img -nographic

# Options utiles
-m 512M              # RAM allouée
-smp 2               # Nombre de CPU
-nographic           # Pas de fenêtre graphique, console série
-s                   # Serveur GDB sur localhost:1234
-S                   # Pause au démarrage (attendre GDB)
-enable-kvm          # Activer la virtualisation matérielle
```

# QEMU — déboguer le noyau avec GDB

```
# Terminal 1 : lancer QEMU avec serveur GDB
qemu-system-x86_64 -kernel bzImage -initrd initramfs.cpio.gz \
    -append "console=ttyS0 nokaslr" -nographic -s -S

# Terminal 2 : attacher GDB au noyau
gdb vmlinux
(gdb) target remote localhost:1234
(gdb) b start_kernel        # Point d'arrêt dans l'init noyau
(gdb) continue
```

> **`nokaslr`** désactive la randomisation des adresses du noyau — indispensable pour que GDB retrouve les symboles aux bonnes adresses.

# GRUB — configuration minimale

```
# /boot/grub/grub.cfg

serial
terminal_input  serial
terminal_output serial

set root=(hd0,1)

menuentry "Mon Linux" {
    linux  /boot/vmlinuz root=/dev/sda1 console=ttyS0
    # initrd /boot/initramfs.cpio.gz   # si nécessaire
}
```

- `(hd0,1)` : premier disque, première partition (numérotation GRUB)
- `linux` : charge le noyau
- `initrd` : charge l'initramfs (optionnel si le rootfs est direct)

# Résumé : ce dont on a besoin pour le TP

| Élément | Rôle | Commande clé |
| --- | --- | --- |
| `bzImage` | Image noyau démarrable | `make` dans les sources Linux |
| `initramfs.cpio.gz` | Rootfs initial en RAM | `find \| cpio \| gzip` |
| `busybox` | Espace utilisateur minimal | `make defconfig && make install` |
| `/init` | 1er programme exécuté | Script shell ou binaire |
| QEMU | Émulateur pour tester | `qemu-system-x86_64` |